ToggleMaster

Roteiro de gravação

Da observabilidade à recuperação automática

21 minutos de roteiro
+ 4 minutos de margem

Em cada cena: tela, ações, sua fala e a evidência que precisa aparecer.

Use o PowerPoint como apoio ao apresentador. O vídeo deve demonstrar as ferramentas operando, e não apenas percorrer slides.

Fonte: enunciado FIAP, páginas 5 e 6.
Limite oficial: até 25 minutos. Edição: 2026-11-09.

Antes de apertar REC

Checklist de preparação - fora do tempo de gravação

1. Brave: GitHub (README, código, Actions e self-healing), ArgoCD, Grafana (dashboard, Explore/Loki e Alert rules), New Relic (Maps e Distributed tracing), PagerDuty (Incidents) e Slack (#incidentes-tech-challenge).
2. Lens: Nodes e Pods do cluster correto. É uma interface de inspeção; a prova de GitOps fica no ArgoCD. AWS Console é apoio opcional.
3. Terminais: A para tráfego; B para pods em watch; C para requisição e falha. Mantenha os túneis 8080, 3000 e 8004 e o tráfego contínuo ativos.
4. Faça login antes. Grave 20 segundos de teste de áudio e reproduza. Feche notificações pessoais e esconda senhas, .env, Secrets, tokens e páginas de chaves.
5. Ensaie a cadeia completa. Antes de repetir, aguarde o cooldown de 600 segundos, confira HTTP 200 e alerta Normal. Comandos detalhados em COMANDOS.md.

As falas são sugestões. Afirme sucesso somente quando a tela mostrar o resultado. Um teste manual não comprova a reação automática.

01 Apresente o problema

00:00 - 01:00 | GitHub · README

O QUE MOSTRAR

1. Mostre o nome ToggleMaster e os cinco serviços.
2. Aponte o diagrama de observabilidade.
3. Explique o objetivo antes das ferramentas.

SUA FALA

Este é o ToggleMaster, uma plataforma de feature flags com cinco microsserviços. A Fase 4 evolui a base das fases anteriores para responder a três perguntas: o sistema está saudável, onde uma requisição falhou e como reagimos ao incidente? Mantivemos Terraform, CI/CD com verificações de segurança e GitOps. Agora acrescentamos métricas, logs, traces e uma tentativa automática de recuperação. Vou demonstrar o ambiente funcionando e depois provocar uma falha controlada para acompanhar a resposta completa.

O avaliador entende o problema e o que será demonstrado.

Fonte: Enunciado FIAP, p. 2–3. Contextualização sugerida.

02 Mostre a base automatizada

01:00 - 03:00 | GitHub · código + Actions

O QUE MOSTRAR

1. Abra terraform/: base, cluster e k8s.
2. Abra um CI de serviço concluído na main.
3. Mostre testes, lint, SAST/SCA, imagem e publicação.

SUA FALA

A infraestrutura está dividida em três camadas Terraform, com estados separados. O código fica no Git; o state fica no backend S3 e relaciona os recursos declarados aos recursos existentes. Os cinco serviços possuem pipelines que reutilizam verificações por linguagem. Aqui vemos build e testes, análise do código e das dependências. No pull request, a imagem também é construída e verificada. No push da main, o pipeline publica no ECR usando OIDC e atualiza a tag no GitOps. É o ArgoCD que reconcilia essa configuração no cluster.

Abra o run correto; verde de CI não substitui a prova do cluster.

Fonte: Enunciado FIAP, p. 2 e 5; .github/workflows; terraform/.

03 Prove EKS + GitOps

03:00 - 05:00 | Lens · Nodes/Pods + ArgoCD

O QUE MOSTRAR

1. Lens: mostre os dois nós Ready e os workloads.
2. ArgoCD: mostre as seis Applications Synced/Healthy.
3. Abra observability e explique os charts Helm.

SUA FALA

Este é o cluster EKS provisionado por código. No Lens, vemos os nós e os pods das aplicações e da observabilidade. O Lens é apenas uma interface de inspeção. No ArgoCD, as seis aplicações estão sincronizadas com o estado desejado no Git e saudáveis nesta consulta. A aplicação observability organiza os componentes: kube-prometheus-stack, Loki, Collector gateway e Collector agents. Os valores dos charts e as configurações do dashboard e dos alertas estão versionados. Assim, a stack de monitoramento também segue GitOps.

Mostrar Ready, Running e Synced/Healthy; não apenas arquivos YAML.

Operação: kubectl get nodes / kubectl get pods -n togglemaster / kubectl get pods -n observability / kubectl get applications -n argocd

Fonte: Enunciado FIAP, p. 3 e 5.

04 Leia a saúde no Grafana

05:00 - 08:00 | Grafana · ToggleMaster - Visao geral

O QUE MOSTRAR

1. Mostre requisições/s, erros 5xx e latência p95.
2. Role para CPU/memória dos nós e dos pods.
3. Mostre o painel de logs com dados recentes.

SUA FALA

Este dashboard foi customizado para o ToggleMaster. A taxa de requisições mostra o tráfego recebido pelos serviços HTTP. A proporção de erros indica quantas respostas falharam, e a latência p95 ajuda a observar a experiência nas requisições mais lentas. Mais abaixo, CPU e memória permitem relacionar comportamento da aplicação e recursos do cluster. O painel de logs consulta o Loki. Analytics processa eventos de forma assíncrona, portanto não deve ser apresentado como se tivesse o mesmo tráfego HTTP dos demais serviços. Vou conferir o intervalo de tempo e a atualização para demonstrar dados atuais.

PRINT 01: dashboard com dados; use duas imagens para incluir recursos e logs.

Fonte: Enunciado FIAP, p. 3, 5–6; dashboard versionado.

05 Conecte logs, métricas e traces

08:00 - 10:00 | Grafana Explore · Loki + diagrama OTel

O QUE MOSTRAR

1. No Explore, selecione Loki e execute a consulta.
2. Aponte serviço, timestamp e trace_id de um log.
3. Explique agent → gateway → backends.

SUA FALA

Métricas resumem valores ao longo do tempo; logs registram eventos; traces mostram o caminho de uma execução, dividido em spans. Aqui consulto logs centralizados pelo Loki. O trace_id permite correlacionar um evento ao rastreamento da requisição. Em cada nó, um Collector agent lê os logs dos contêineres e os envia ao gateway. As aplicações enviam seus traces ao mesmo gateway. Ele processa os dados, encaminha logs ao Loki, traces ao New Relic e gera métricas de aplicação pelo spanmetrics para o Prometheus. As métricas de infraestrutura são coletadas diretamente pelo Prometheus nos exporters.

A consulta precisa retornar logs. Explique o caminho real, incluindo o scrape direto.

Operação: {k8s_namespace_name="togglemaster"} |= "flag-service"

Fonte: Enunciado FIAP, p. 3–5; otel-agent.yaml e otel-gateway.yaml.

06 Siga uma requisição no APM

10:00 - 13:00 | New Relic · Maps + Distributed tracing

O QUE MOSTRAR

1. Mostre o mapa com os cinco serviços e dependências.
2. Faça uma requisição /evaluate no terminal.
3. Abra o trace correspondente; expanda os spans.

SUA FALA

Escolhemos New Relic como o APM comercial desta entrega. Ele recebe os traces via OpenTelemetry. No mapa, observamos as dependências dos cinco microsserviços. Agora faço uma requisição ao evaluation-service. No trace detalhado, vemos as chamadas aos serviços de flags, segmentação e autenticação, além das dependências de dados. O caminho varia: um acerto no Redis pode encurtar a execução. Para a demonstração completa, seleciono uma requisição sem cache e confiro seu horário. O fluxo de SQS e analytics é assíncrono; a resposta HTTP não espera a persistência analítica. A duração de cada span ajuda a localizar uma operação lenta.

PRINT 02: trace aberto com ID, serviço de entrada, spans e durações; a lista de traces não basta.

Operação: Ver COMANDOS.md: crie uma flag exclusiva antes de testar /evaluate.

Fonte: Enunciado FIAP, p. 4 e 6.

07 Provoque a falha controlada

13:00 - 15:00 | Terminal · tráfego + flag-service

O QUE MOSTRAR

1. Mostre HTTP 200 antes da falha.
2. Ative /admin/fault usando o comando preparado.
3. Mostre HTTP 503; mantenha tráfego contínuo.

SUA FALA

Agora vou provocar uma falha transitória controlada no flag-service. O objetivo é demonstrar detecção e reação automáticas, sem desligar um banco. A rota administrativa exige autenticação e faz as rotas de flags responderem 503. O health check continua saudável; por isso, o Kubernetes não reinicia o processo apenas pela liveness probe. O tráfego contínuo permite medir a proporção de erros. A falha fica na memória do processo e desaparece quando um novo pod assume. Vou observar a cadeia sem clicar em Run workflow nem reiniciar manualmente o serviço.

Grave a causa e o efeito; falha manual → resposta automática.

Operação: kubectl logs -f trafego -n togglemaster / # Em outro terminal: ver COMANDOS.md, seção Falha controlada.

Fonte: Enunciado FIAP, p. 5–6; services/flag-service/app.py.

08 Acompanhe alerta e comunicação

15:00 - 17:00 | Grafana Alerting → PagerDuty → Slack

O QUE MOSTRAR

1. Mostre FlagService5xxAlto em Firing.
2. Abra o incidente criado automaticamente no PagerDuty.
3. Mostre a mensagem detalhada no canal do Slack.

SUA FALA

A regra avalia a proporção de erros 5xx do flag-service. O limite escolhido é maior que dez por cento durante um minuto; isso evita reagir a um único erro isolado. O exemplo do enunciado usa outro serviço e outro percentual, mas nossa regra atende ao mesmo objetivo. Quando entra em Firing, o Grafana abre um incidente no PagerDuty e envia o contexto ao Slack. PagerDuty organiza o incidente e o acionamento humano; Slack concentra a comunicação. Aqui confiro serviço, horário e descrição para mostrar que as telas correspondem à mesma falha. Um DatasourceError não é a evidência desta regra de erros HTTP.

PRINT 03: notificação do incidente real; capture também Firing e PagerDuty como apoio.

Fonte: Enunciado FIAP, p. 5–6; rules.yaml, policies.yaml e contactpoints.yaml.

09 Prove a correção automática

17:00 - 20:00 | GitHub Actions · Self-healing + terminal

O QUE MOSTRAR

1. Abra o run disparado por repository_dispatch.
2. Mostre o passo de restart e resultado restarted.
3. Mostre pod novo, HTTP 200 e alerta resolvido.

SUA FALA

O mesmo alerta chama o webhook do GitHub e dispara esta execução automaticamente. O workflow assume uma role temporária por OIDC. Antes de agir, o script valida o serviço em uma lista permitida e verifica um intervalo mínimo de dez minutos entre reinícios. Ele executa rollout restart e aguarda o rollout terminar. Aqui vemos o registro da ação e a substituição do pod. O tráfego volta a 200, o alerta retorna ao normal e o incidente é resolvido. Isso é uma tentativa de mitigação, não uma cura universal: um banco indisponível, por exemplo, não seria corrigido apenas reiniciando a aplicação.

PRINT 04: log/execução da automação, com evento automático e reinício concluído.

Operação: kubectl get pods -n togglemaster -l app=flag-service -w

Fonte: Enunciado FIAP, p. 5–6; self-healing.yml e restart.sh.

10 Feche com as evidências

20:00 - 21:00 | README + relatório PDF

O QUE MOSTRAR

1. Recapitule falha → detecção → comunicação → recuperação.
2. Mostre onde estão código, relatório e evidências.
3. Confirme identificação e link acessível do vídeo na entrega.

SUA FALA

Demonstramos a base automatizada, as métricas e os logs centralizados, o roteamento pelo Collector e o tracing no APM. Na prova do incidente, acompanhamos a falha controlada, o alerta, a comunicação e a tentativa automática de recuperação. O repositório reúne o código de infraestrutura, a instrumentação e a automação. O relatório traz os quatro prints exigidos e justifica a arquitetura e as escolhas de New Relic e PagerDuty. As evidências precisam corresponder ao que foi realmente executado. Obrigado.

Finalize até 25 minutos. Meta do roteiro: 21 minutos + 4 de margem.

Fonte: Enunciado FIAP, p. 5–6.

Feche a entrega com evidências

O relatório PDF exige quatro prints - enunciado, p. 6.

1. Dashboard Grafana: requisições, recursos do cluster e logs com dados. Divida em duas imagens se necessário.
2. Trace no APM: requisição de entrada, ID, operações e durações. Uma lista de traces não basta.
3. ChatOps: notificação detalhada do incidente real, com horário e serviço.
4. Self-healing: execução/log da automação com o disparo automático e a ação corretiva.

ONDE DEPOSITAR COMPLEMENTOS

docs/evidencias/entrada-local/
Entrada local ignorada pelo Git. As capturas só entram na pasta pública datada depois da revisão.

ANTES DE SUBMETER

Confirme nomes, RMs, usernames, links do repositório e do vídeo; as justificativas OTel, New Relic e PagerDuty; áudio legível, vídeo até 25 minutos e acesso do avaliador. Não há garantia de nota: qualquer requisito sem demonstração clara pode perder pontos (p. 5).